

ECSPros

Kod Optimizasyon Raporu

API, PostgreSQL ve Redis'in ayrı sunucularda çalışacağı dağıtık mimariye göre hazırlanmış, yalnız uygulama koduna odaklanan performans ve ölçeklenebilirlik planı.

Kapsam: Storefront, ürün listeleme, ana sayfa kompozisyonu, cache kullanımı ve Razor çıktı maliyeti
Hedef: 4.000 eşzamanlı aktif kullanıcı için ölçülebilir ve güvenli kod iyileştirmeleri
Tarih: 15 Ağustos 2026

1. Yönetici özeti

Projenin modüler yapısı, ortak `NpgsqlDataSource` kullanımı, sorgulardaki `AsNoTracking` yaklaşımı ve Redis arızasında uygulamayı ayakta tutan devre kesici tasarımı sağlam bir temel oluşturuyor. Buna karşılık sıcak kullanıcı yollarında fazla sayıda ardışık veri erişimi bulunuyor.

PostgreSQL ve Redis API sunucularından ayrı çalışacağı için asıl hedef sorguları yalnız hızlandırmak değil, her HTTP isteğinde yapılan ağ gidiş-dönüşü sayısını azaltmaktır.

En büyük darboğaz

`GetStoreProductsQuery` tek sayfa için fiyat, stok, görsel, kampanya, yorum ve sayaç verilerini art arda topluyor.

İkinci darboğaz

`PageComposer` ana sayfa bloklarını seri çözüyor ve Redis cache hit olsa bile aktif versiyon için DB'ye gidiyor.

Ölçek riski

Fiyat, metrik ve arama sıralamalarında tüm adaylar uygulama belleğine alınabiliyor.

İlk hedef

Sayfalama öncesi minimum veri, sayfalama sonrası yalnız görünen ürünler için toplu kart zenginleştirilmesi.

Önerilen öncelik

Sıra	Çalışma	Beklenen etki	Risk
1	Ürün sorgusunu iki aşamalı hale getirmek	Çok yüksek	Orta
2	Bellekte sıralamayı PostgreSQL/read-model tarafına taşımak	Çok yüksek	Orta-yüksek
3	Ana sayfa cache hit yolundan DB sorgusunu kaldırmak	Yüksek	Düşük-orta
4	Cache stampede koruması eklemek	Ani yükte çok yüksek	Orta
5	Küçük ürün kart DTO'su ve daha küçük HTML	Yüksek	Düşük-orta

2. P0 - Ürün listeleme sıcak yolunun yeniden düzenlenmesi

P0 Platformun bütün fiyatlarını çekme

`StorefrontChannelPricingService.GetActiveVariantPricesAsync` platformdaki bütün aktif varyant fiyatlarını yükleyip sözlüğe dönüştürüyor. Kullanıcı yalnız 24 ürün görecektir olsa bile katalogdaki tüm fiyatlar API belleğine taşınabiliyor.

Değişiklik: Önce sayfadaki ürün ve varyant kimlikleri belirlenmeli; ardından fiyat servisine yalnız bu kimlikler verilmelidir.

```
GetActiveVariantPricesAsync(  
    Guid firmPlatformId,  
    IReadOnlyCollection<Guid> variantIds,  
    CancellationToken ct)
```

Etki: DB'den taşınan satır, allocation ve serialization miktarında büyük azalma.

P0 İki aşamalı listeleme

Önerilen sorgu akışı:

1. Filtre + sıralama + sayfalama
Sonuç: ProductId listesi ve toplam kayıt
2. Yalnız sayfadaki ProductId/VariantId kümesi için:
fiyat + stok + görsel + özellik + kampanya + yorum + sayaç

Stok, kampanya, yorum ve sosyal kanıt servisleri de ID listesi kabul eden toplu metotlara dönüştürülmelidir. Kart verileri ürün başına çağrı ile değil, sayfa başına toplu çağrı ile çözülmelidir.

P0 Bellekte sıralamayı kaldırma

Fiyat, metrik ve arama sıralamalarında tüm adayların `ToListAsync` ile belleğe alınması katalog büyüdükçe CPU ve RAM kullanımını artırır.

Sıralama ve sayfalama PostgreSQL tarafında yapılmalıdır:

```
ORDER BY effective_price, product_id  
OFFSET @offset  
LIMIT @pageSize
```

Efektif fiyat, puan, yorum, favori, sepet, görüntülenme, satış ve stok durumunu taşıyan ürün arama read-model'i uzun vadede en verimli çözümdür.

Etki: Katalog büyüklüğünden bağımsız, daha kararlı API RAM ve CPU kullanımı.

Önemli: Yalnız `Task.WhenAll` kullanmak sorgu süresini kısaltabilir, fakat DB yükünü azaltmaz. Önce veri miktarı ve sorgu sayısı düşürülmelidir.

3. P0 - Ana sayfa kompozisyonu ve cache

P0 Redis cache hit yolundan DB'yi çıkarma

`PageComposer.ComposeAsync`, cache anahtarını üretmek için önce PostgreSQL'den aktif snapshot versiyonunu alıyor. Böylece hazır içerik Redis'te bulunsa bile her istekte DB çağrısı oluyor.

Aktif pointer Redis ve kısa süreli local memory katmanında tutulmalıdır:

```
page-active:{platformId}
{
  "version": 12,
  "snapshotId": "...",
  "publishedAt": "..."
}
```

Sayfa yayınlandığında pointer güncellenir. Versiyonlu paketler immutable olduğu için API node'larının local memory cache kullanması güvenlidir.

P0 Blokları batch veya sınırlı paralel çözme

Ürün, koleksiyon ve tab blokları mevcut akışta seri çözülüyor. Kısa vadede bağımsız bloklar en fazla 2-3 eşzamanlı işlemle çözülebilir. Her işlem kendi DI scope ve `DbContext` instance'ını kullanmalıdır.

```
Blokları grupla
├ ürün kaynakları → batch çöz
├ koleksiyon kaynakları → batch çöz
└ statik bloklar → doğrudan map et
```

Aynı scoped `DbContext` paralel görevlerde kullanılmamalıdır.

P0 Cache stampede koruması

Cache süresi dolduğunda aynı anahtar isteyen yüzlerce istek pahalı sorguyu eşzamanlı başlatabilir. Key bazlı request coalescing uygulanmalıdır.

- API process içinde key bazlı `SemaphoreSlim`.
- Node'lar arasında kısa süreli Redis lock.
- Uygunsa eski veriyi kısa süre sunan stale-while-revalidate.
- Lock sahibinin hata vermesi durumunda güvenli süre aşımı.

Etki: Cache soğuması ve kampanya başlangıcı gibi ani trafik anlarında PostgreSQL'in korunması.

P1 Yayın sırasında hazır vitrin paketi

Kalıcı çözüm, içerik yayınlandığında anonim ziyaretçi için hazır vitrin paketinin oluşturulmasıdır. İstek anında blok konfigürasyonu ve değişmeyen içerikler tekrar çözülmemelidir. Fiyat ve stok gibi değişken alanlar ayrı kısa ömürlü read-model ile birleştirilebilir.

4. P1 - Kart zenginleştirme ve DTO küçültme

P1 Ana sayfa için küçük kart DTO'su

Mevcut ürün DTO'su renkler, özellikler, galeri, bedenler, video, kampanyalar ve sosyal sayaçlar gibi çok sayıda alan taşıyor. Ana sayfa için yalnız gerçekten gösterilen alanlar gönderilmelidir.

```
StorefrontProductCardDto
  Id, Code, Name
  ImageUrl
  Price, CompareAtPrice
  Rating, ReviewCount
  Badge
```

Ürün detayına özgü renk, beden, galeri ve kapsamlı özellikler detay sorgusunda kalmalıdır.

Etki: Daha küçük SQL sonucu, Redis paketi, JSON ve Razor HTML çıktısı.

P1 Kampanya ve sosyal sayaç cache'i

Aktif kampanya listesi platform bazında 30-60 saniye cache'lenebilir ve kampanya değişikliğinde invalidation yapılabilir. Favori, sepet ve görüntülenme gibi sosyal kanıt değerleri için 15-30 saniyelik cache kullanıcı deneyimi açısından yeterlidir.

Sipariş toplamı, gerçek stok rezervasyonu ve ödeme gibi doğruluk gerektiren değerler bu cache yaklaşımına dahil edilmemelidir.

P1 Stok verisini istenen varyantlarla sınırlandırma

InStockProductProvider tüm stoklu ürün veya varyant ID'lerini belleğe alıyor. Kart oluşturma sırasında yalnız sayfadaki varyantların stok durumu sorgulanmalıdır.

```
GetInStockVariantIdsAsync(
  IReadOnlyCollection<Guid> variantIds,
  CancellationToken ct)
```

Tüm katalog için stok filtresi gereken durumlarda ayrı, önceden güncellenen read-model kullanılabilir.

P1 Koleksiyon üye N+1 sorgusu

VitrinVmBuilder koleksiyondaki her farklı üye için ayrı çağrı yapıyor. Üye kimliklerini toplu kabul eden bir servis eklenip tek sorguda adlar alınmalıdır.

5. P1/P2 - Storefront ve render maliyeti

P1 Ortak sayfa hazırlığını güvenli paralelleştirme

Platform ve kullanıcı bağlamı çözüldükten sonra navigasyon, yasal belgeler, footer ve global duyuru birbirinden bağımsız olarak başlatılabilir. Servislerin aynı `DbContext` instance'ını paylaşmadığı doğrulanmalıdır.

P1 Başlangıç HTML'ini küçültme

Lokal ölçümde ana sayfa HTML'i yaklaşık 422 KB seviyesindeydi. İlk viewport ve SEO açısından önemli bloklar SSR kalmalı; aşağıdaki bloklar continuation endpoint ile yüklenmelidir.

- İlk açılışta blok başına 8-12 ürün.
- Ekran altındaki bloklar lazy-load.
- Modal şablonları gerektiğinde yükleme.
- Tekrarlanan inline scriptleri ortak JS modüllerine taşıma.

P2 Snapshot JSON'unu bir kez derleme

Blok config JSON'ları resolver ve view-model builder içinde tekrar parse ediliyor. Snapshot yayınlanırken strongly typed config oluşturulup Redis paketine yazılabilir.

P2 Redis payload optimizasyonu

Önce DTO küçültülmelidir. Büyük paketlerde daha sonra byte tabanlı serialization ve yalnız belirli boyut üzerindeki payload'lar için sıkıştırma değerlendirilebilir. Küçük kayıtları sıkıştırmak gereksiz CPU maliyetidir.

Korunması gereken iyi yaklaşım: Redis erişilemediğinde cache servisinin uygulamayı düşürmemesi doğru. Optimizasyon yapılırken cache hiçbir zaman iş doğruluğunun tek kaynağı haline getirilmemelidir.

6. Uygulama planı

Paket 1 - Ürün listeleme sıcak yolu

1. Mevcut ürün kartı çıktısı için karakterizasyon testleri yaz.
2. Fiyat servisini yalnız sayfadaki varyantları alacak şekilde genişlet.
3. Stok, görsel ve video sorgularını sayfadaki ID kümesiyle sınırlandır.
4. Kampanya ve sosyal sayaç için kısa TTL cache ekle.
5. Ana sayfa için küçük kart DTO'su oluştur.

Paket 2 - Sıralama ve read-model

1. Fiyat sıralamasını DB tarafına taşı.
2. Metrik sıralamaları için ürün arama read-model'i oluştur.
3. Arama puanlamasını SQL/read-model üzerinde sayfala.
4. Sonuç eşitliğini eski uygulama ile otomatik karşılaştır.

Paket 3 - Ana sayfa kompozisyonu

1. Aktif snapshot pointer'ını cache'e al.
2. Key bazlı cache stampede koruması ekle.
3. Blok kaynaklarını batch çöz.
4. Koleksiyon üye sorgusunu toplu hale getir.
5. Yayın sırasında hazır vitrin paketi üret.

Paket 4 - Render ve payload

1. İlk HTML'de gösterilen ürün sayısını azalt.
2. Alt blokları lazy-load yap.
3. Tekrarlanan JSON parse ve inline scriptleri azalt.
4. Redis payload boyutlarını ölçüp yalnız gerekirse sıkıştır.

7. Test ve kabul kriterleri

Her deęişiklikten önce ve sonra aynı veri setiyle ölçüm yapılmalıdır. Ortalama deęer tek başına yeterli deęildir; P95 ve P99 izlenmelidir.

Ölçüm	Kabul yaklaşımı
DB sorgu sayısı	Ana sayfa ve ürün listesi için istek başına sorgu sayısı kayıt altına alınmalı; her paket sonunda düşüş gösterilmeli.
DB toplam süresi	Sorgu sayısı kadar, sorguların toplam elapsed süresi de karşılaştırılmalı.
Warm-cache	Hazır vitrin cache'inde PostgreSQL çağırısı yapılmadığı doğrulanmalı.
Cold-cache	Eşzamanlı isteklerde pahalı üretimin tek kez çalıştığı doğrulanmalı.
Response boyutu	HTML, JSON ve Redis payload boyutları ölçülmeli.
API kaynakları	Allocation, GC, CPU ve peak memory karşılaştırılmalı.
Latency	Ana sayfa, kategori, arama, fiyat sıralaması ve ürün detayı için P50/P95/P99 ölçülmeli.
Doğruluk	Fiyat, stok, kampanya, sıralama ve sayfalama eski davranışla karşılaştırılmalı.

Önerilen test senaryoları

- Ana sayfa: cold-cache ve warm-cache.
- Kategori ilk sayfa ve devam sayfaları.
- Fiyat artan/azalan sıralama.
- Çok kelimeli arama ve renk eşleşmesi.
- Kampanyalı ve stokluz ürünler.
- Redis kısa süre kullanılmadığında doğru fallback.
- Aynı cache anahtarına yüksek eşzamanlı istek.

Karar: İlk kod deęişikliği `getStoreProductsQuery` içindeki bütün platform verisini yükleme davranışını kaldırmak olmalıdır. En hızlı, ölçülebilir ve düşük riskli performans kazancı bu noktadadır.

8. Dosya bazlı çalışma listesi

Dosya / bileşen	Önerilen değişiklik	Öncelik
GetStoreProductsQuery.cs	İki aşamalı sayfalama, bellekte sıralamanın kaldırılması, toplu enrichment.	P0
StorefrontChannelPricingService.cs	Yalnız verilen varyantların fiyatlarını getiren overload.	P0
InStockProductProvider.cs	Sayfadaki varyantlara göre stok sorgusu.	P1
PageComposer.cs	Aktif pointer cache, stampede koruması, batch blok çözümü.	P0
PageBlockSourceResolver.cs	Kaynakları tek tek değil gruplu/batch çözme.	P0
VitrinVmBuilder.cs	Koleksiyon üyelerini toplu yükleme, küçük DTO.	P1
StorePageController.cs	Bağımsız sayfa hazırlıklarını güvenli paralelleştirme.	P1
ProductCampaignResolver.cs	Platform bazlı kısa TTL cache ve invalidation.	P1
RedisCacheService.cs	Request coalescing, stale seçenekleri, büyük paket optimizasyonu.	P1/P2
Razor storefront view'ları	Daha az başlangıç kartı, lazy-load, daha az inline script.	P2

9. Sonuç

Bu proje için en önemli optimizasyon daha fazla cache eklemek değil, her istekte üretilen veri miktarını ve PostgreSQL ağ turu sayısını azaltmaktır. Önerilen sıra; önce ürün listeleme sıcak yolunu küçültmek, sonra sıralamayı read-model'e taşımak, ardından ana sayfa kompozisyonunu cache-hit sırasında DB'siz çalıştırmaktır.

Bu yaklaşım iki API node, uzak PostgreSQL ve ortak Redis mimarisinde kararlı performans sağlar; aynı zamanda kodun doğruluğunu yalnız cache'e bağımlı hale getirmez.

Rapor yalnız kod optimizasyonlarını kapsar. Altyapı ve sunucu yapılandırması ayrı çalışma olarak ele alınmalıdır.